

AB测试

意义：

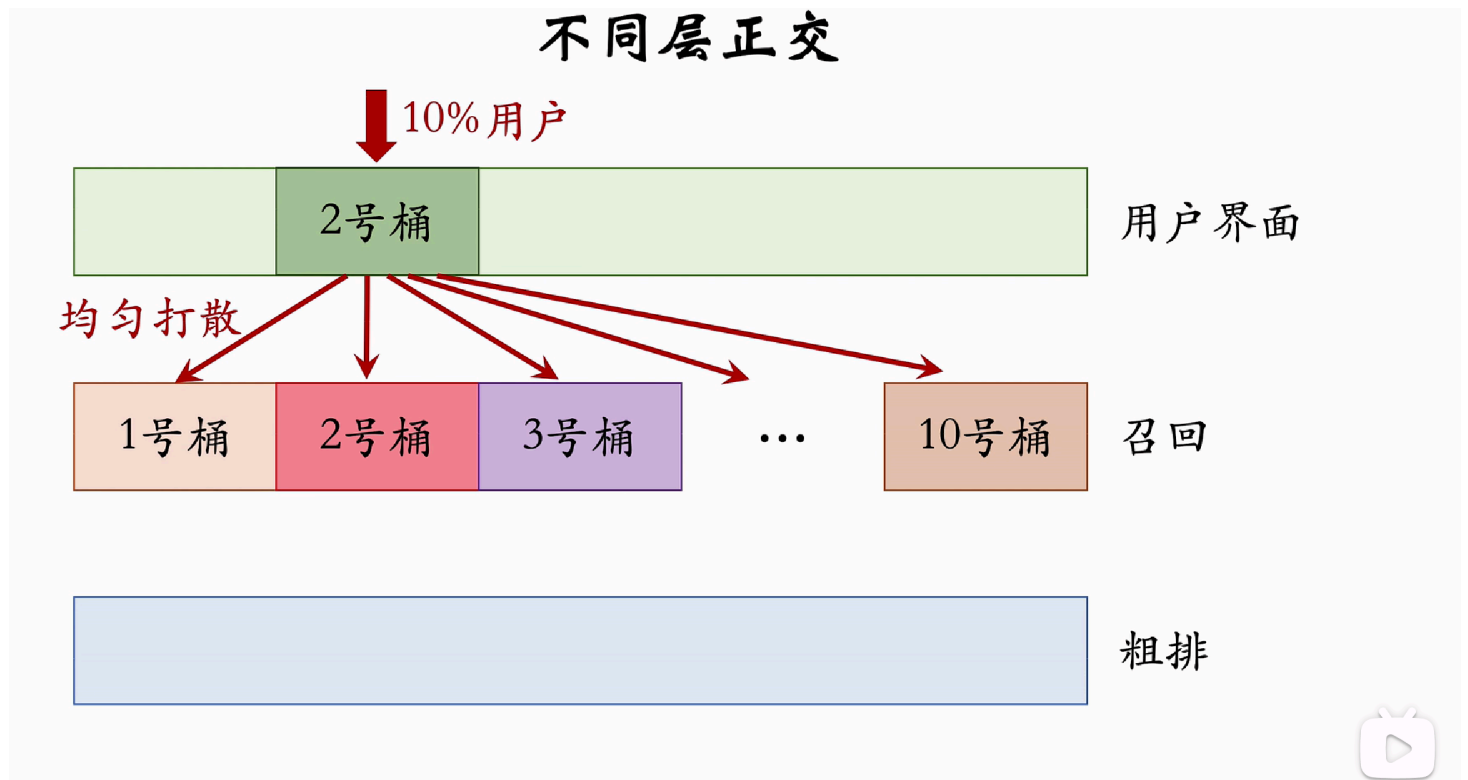
1.做线上小流量A/B测试，考察新方法对线上指标的影响

2.通过A/B测试，选取最优参数

实验组指标显著优于对照组，则值得推全

随机分桶（哈希分类）

分层实验：同层互斥，不同层正交



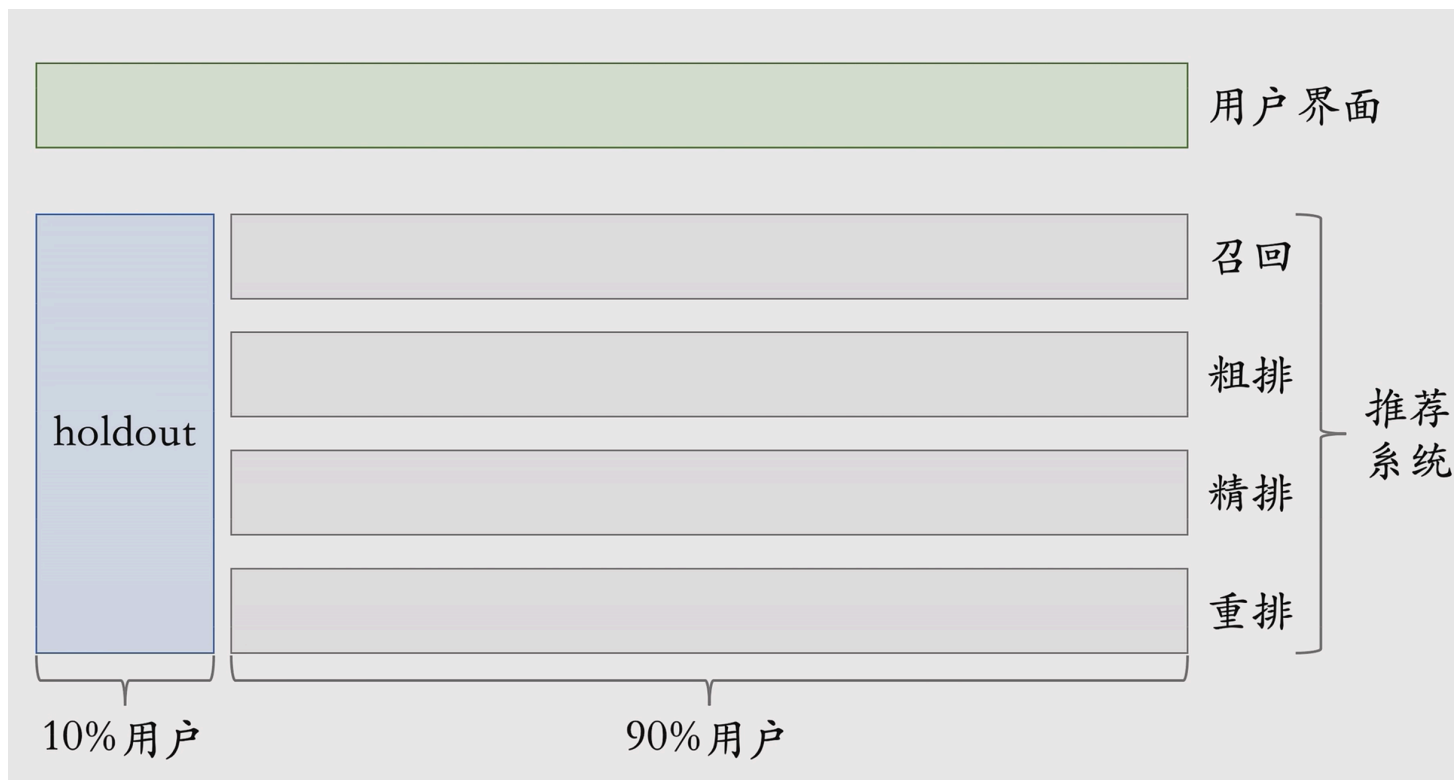
同类的策略天然互斥，可能互相干扰，对于一个用户只能用其中一种。

不同类型的策略通常不会相互干扰，可以正交。

Holdout机制

取10%用户作为holdout桶，90%的用户做实验，两者互斥。

将归一化后的两者指标做对比



反转实验

有的指标有滞后性，需要长期观测

反转实验既可以尽快推全，也可以长期观测实验指标。

具体做法：在推全的新层中开一个旧策略的桶，长期观测实验指标

协同过滤算法

根据用户的喜好和其他兴趣相近的用户推荐物品

应用最广泛的：基于邻域的方法

- UserCF 推荐相似用户喜欢的产品
- ItemCF 推荐他之前喜欢的物品相似的物品

常用的相似度

- Jaccard相关系数
- 余弦相似度
- 皮尔逊相关系数，相比于余弦相似度，通过用户平均分来减小偏置影响

UserCF

```
user_data=loadData()
similarity_matrix=pd.DataFrame(
    np.identity(len(user_data))
    #创建n x n单位矩阵
    index=user_data.keys()
    columns=user_data.keys()
    #行和列索引为用户_data的标识符
)
for u1,items1 in user_data.items():
    for u2,items2 in user_data.items():
        if u1==u2:
            continue
        vec1,vec2=[],[]
        for item,rating1 in items1.items():
            rating2=items2.get(item,-1)
            #用户2的评价中能否找到，找不到返回 -1
            if rating2== -1:
                continue
            vec1.append(rating1)
            vec2.append(rating2)
            # 计算不同用户的皮尔逊相关系数,correlation coefficient的缩写
        similarity_matrix[u1][u2]=np.corrcoef(vec1,vec2)[0][1]

sim_users = similarity_matrix[target_user].sort_values(ascending=False)[1:num+1].index.tolist()
print(f'与用户{target_user}最相似的{num}个用户为: {sim_users}')
#计算目标用户对商品X的预测评分
for user in sim_users:
    corr_value=similarity_matrix[target_user][user]
    user_mean_rating=np.mean(list(user_data[user].values()))

    weighted_scores += corr_value * (user_data[user][target_item]-user_mean_rating)
    corr_values_sum+=corr_value
target_user_mean_rating = np.mean(list(user_data[target_user].values()))
target_item_pred = target_user_mean_rating + weighted_scores / corr_values_sum
```

召回率 预测准确的物品占用户实际喜欢的物品的比例

精确率 对用户准确推荐的物品占总推荐物品的比例

覆盖率 推荐系统能够推荐出来的物品占总物品集合的比例

ItemCF

```
item_data=loadData()
similarity_matrix=pd.DataFrame(
    np.identity(len(item_data))
    index=item_data.keys()
    columns=item_data.keys()
)

for i1,users1 in item_data.items():
    for i2,users2 in item_data.items():
        if i1==i2:
            continue
        vec1,vec2=[],[]
        for user,rating1 in users1.items():
            rating2=users2.get(user,-1)
            if rating2 == -1:
                continue
            vec1.append(rating1)
            vec2.append(rating2)
        similarity_matrix[i1][i2]=np.corrcoef(vec1,vec2)[0][1]

sim_items=similarity_matrix[target_item].sort_values(ascending=False)[1:num+1].index.tolist()
print(f'与物品{target_item}最相似的{num}个物品为: {sim_items}')

for item in sim_items:
    corr_value=similarity[target_item][item]
    user_mean_rating=np.mean(list(item_data[item].values()))
    weighted_score+=corr_value * (item_data[item][target_user] - user_mean_rating)
    corr_value_sum+=corr_value
target_item_pred = target_user_mean_rating + weighted_scores / corr_values_sum
print(f'用户{target_user}对物品{target_item}的预测评分为: {target_item_pred}')
```

协同过滤的缺点

- 泛化能力弱，无法将两个物品相似的信息推广到其他物品的相似性上。
- 热门物品具有很强的头部效应，容易跟大量物品产生相似，而尾部物品由于特征向量稀疏，导致很少被推荐。

什么时候使用UserCF，什么时候使用ItemCF？为什么？

(1) UserCF

所以具备更强的社交特性，这样的特点非常适于用户少物品多，时效性较强的场合。比如新闻推荐场景。此外，UserCF可以发现用户潜在但自己尚未察觉的兴趣爱好。

(2) ItemCF

适用于兴趣变化较为稳定的应用，适合物品少，用户多，用户兴趣固定持久的场合。比如推荐音乐、电影。

Swing算法

之前的Item-based CF 只关注于 Item-User-Item 的路径，即通过物品与用户的交互，来推断物品与物品之间的相似度，从而进行推荐。忽视了噪声，推荐精度存在局限性。

Swing 通过利用 User-Item-User 路径中所包含的信息进行推荐：

若用户 u 和用户 v 之间除了购买过 i 外，还购买过商品 j ，则认为两件商品是具有某种程度上的相似的。也就是说，商品与商品之间的相似关系，是通过用户关系来传递的。如果这两个用户共同购买的物品越少，即这两个用户原始兴趣不相似，但仍同时购买了两个相同的物品，则物品 i 和 j 的相似性越高。

• 计算公式

$$s(i, j) = \sum_{u \in U_i \cap U_j} \sum_{v \in U_i \cap U_j} w_u * w_v * \frac{1}{\alpha + |I_u \cap I_v|}$$

其中 U_i 是点击过商品 i 的用户集合， I_u 是用户 u 点击过的商品集合， α 是平滑系数。

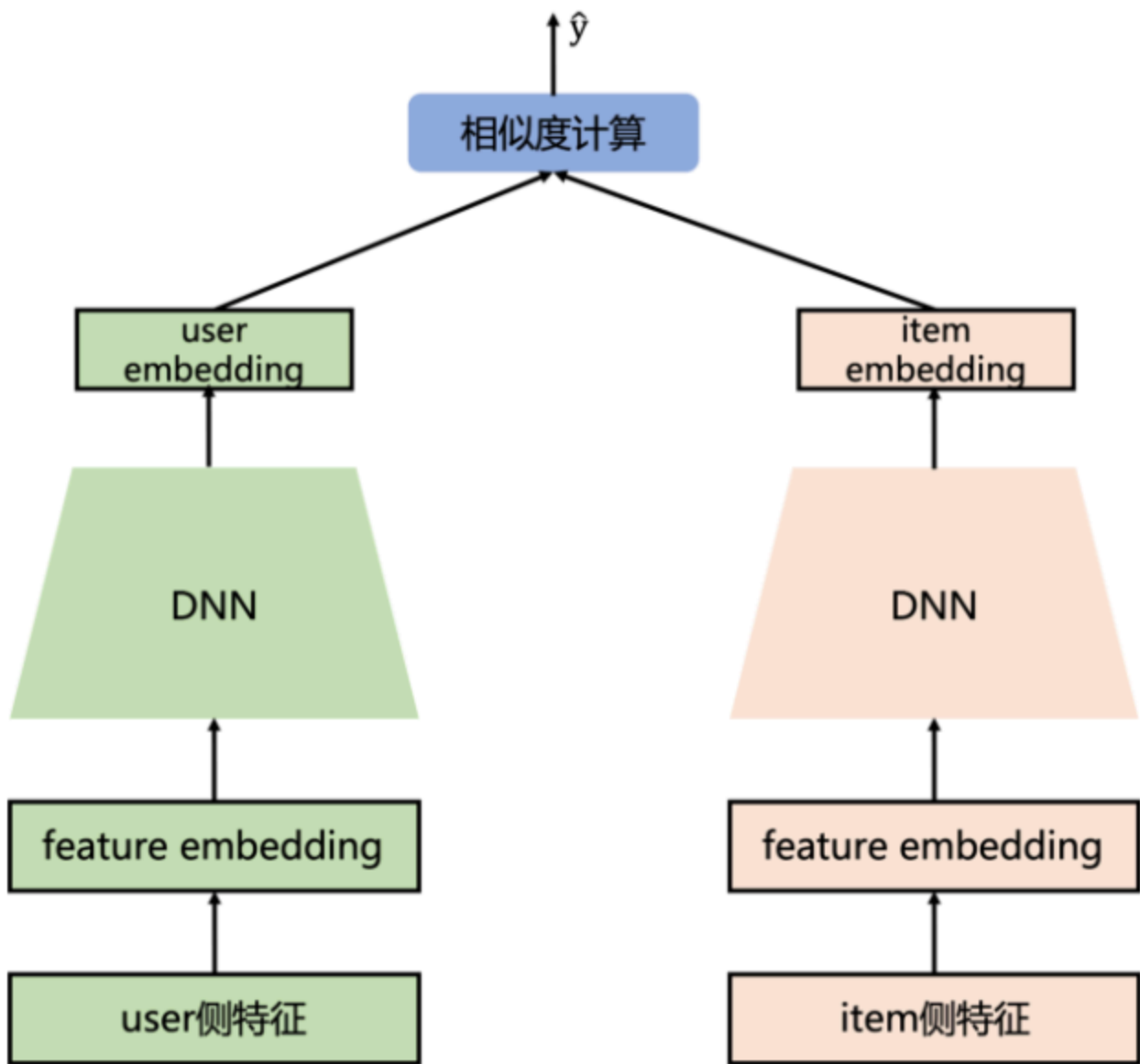
$w_u = \frac{1}{\sqrt{|I_u|}}$, $w_v = \frac{1}{\sqrt{|I_v|}}$ 是用户权重参数，来降低活跃用户的影响。

双塔模型

用户塔：提取用户特征，得到用户表征

物品塔：提取物品特征，得到物品表征

其中ID和离散特征用embedding层处理，连续特征用归一化或分桶做处理



训练方式:

1. Pointwise:

- 把召回看作二元分类任务
- 对于正样本, 鼓励 $\cos(a,b)$ 接近+1
- 对于负样本, 鼓励 $\cos(a,b)$ 接近-1
- 正负样本数量控制在1:2或1:3

2. Pairwise

- 每次输入三元组<用户a, 正样本 b^+ , 负样本 b^- >
- 基本想法: 鼓励 $\cos(a,b^+)$ 大于 $\cos(a,b^-)$

- 超参数M，正负样本的兴趣之差
可以通过最小化以下的损失函数来训练双塔模型

基本想法：鼓励 $\cos(\mathbf{a}, \mathbf{b}^+)$ 大于 $\cos(\mathbf{a}, \mathbf{b}^-)$

Triplet hinge loss:

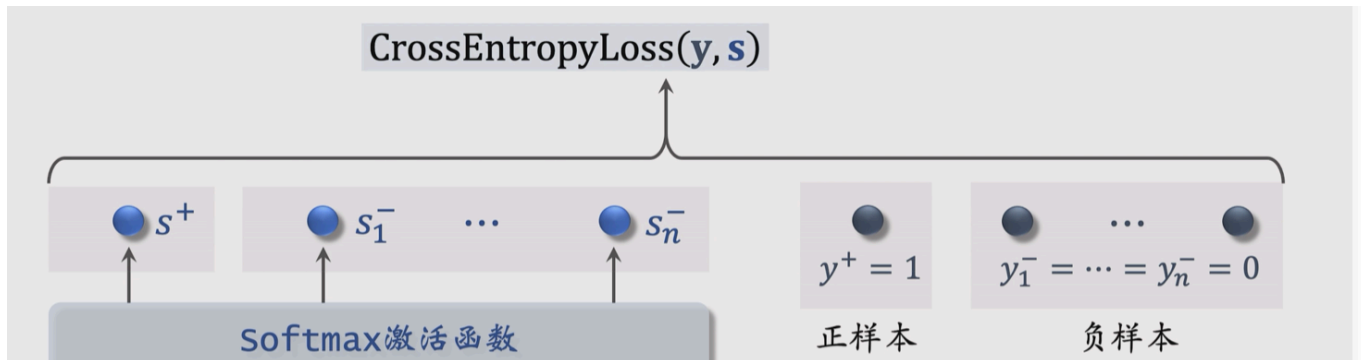
$$L(\mathbf{a}, \mathbf{b}^+, \mathbf{b}^-) = \max\{0, \cos(\mathbf{a}, \mathbf{b}^-) + m - \cos(\mathbf{a}, \mathbf{b}^+)\}.$$

Triplet logistic loss:

$$L(\mathbf{a}, \mathbf{b}^+, \mathbf{b}^-) = \log(1 + \exp[\sigma \cdot (\cos(\mathbf{a}, \mathbf{b}^-) - \cos(\mathbf{a}, \mathbf{b}^+))]).$$

3.Listwise

- 每次输入一个用户，一个正样本，多个负样本
- 同样是鼓励 $\cos(\mathbf{a}, \mathbf{b}^+)$ 尽量大， $\cos(\mathbf{a}, \mathbf{b}^-)$ 尽量小
- 将 $\cos(\mathbf{a}, \mathbf{b}^+)$ 和所有 $\cos(\mathbf{a}, \mathbf{b}^-)$ 输入softmax进行归一化



- 利用交叉熵做损失函数，训练时最小化交叉熵，使得 s^+ 接近1， s^- 接近0

选取正负样本

1. 正样本

曝光且有点击的用户-物品二元组

问题：少部分物品占据大量点击导致正样本多是热门物品

解决方法：对冷门物品过采样（采样多次），对热门物品降采样（抛弃一些样本）

2. 负样本

- 简单负样本：

- i. 全体物品

均匀抽样？

对冷门不公平-因为正样本大多是热门物品，导致负样本中热门物品占比少

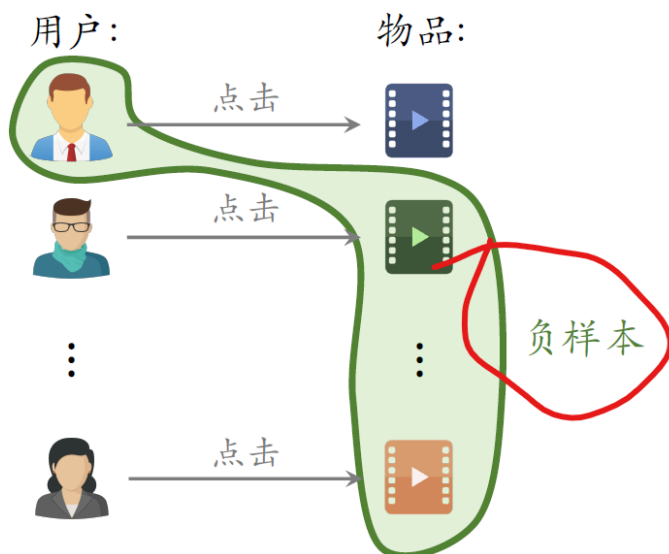
解决方法：非均匀抽样——物品抽样的概率与热门程度正相关

抽样概率 $\propto$ 点击次数^{0.75}

使得热门物品成为负样本的概率大

- ii. batch内负样本

简单负样本：Batch内负样本



- 一个 batch 内有 n 个正样本。
- 一个用户和 $n - 1$ 个物品组成负样本。
- 这个 batch 内一共有 $n(n - 1)$ 个负样本。
- 都是简单负样本。（因为第一个用户不喜欢第二个物品。）

CSDN @冰露可乐

但这种负样本中，抽样概率 $\propto$ 点击次数，热门物品成为负样本的概率过大

解决方法：

将用户的兴趣调整为

$\cos(a, b_i) - \log p_i$ ，其中 $p_i \propto$ 点击次数

- 困难负样本：

- i. 被粗排淘汰的物品（比较困难）

- ii. 在精排中分数排名靠后的物品（非常困难）

- **正样本**：曝光而且有点击。
- **简单负样本**：
 - 全体物品。
 - batch内负样本。
- **困难负样本**：被召回，但是被排序淘汰。
- ➔ • **错误**：曝光、但是未点击的物品做召回的负样本。

曝光说明很感兴趣，导致未点击的因素很多，不能直接用来做负样本

线上服务与更新

离线存储：在训练好双塔模型之后，就可以把模型部署到线上做召回。

把物品塔提取得到的物品的特征，把物品特征向量-物品ID这样的二元组保存到Oracle或者MySQL这样的向量数据库。

在线召回：用户塔不事先计算和存储用户向量，而是当用户发起推荐请求的时候，调用神经网络在线上线算一个特征向量a，

然后把向量a作为query去数据库中做检索查找最近邻。

为什么区分对待？

- 线上算物品向量代价过大，每次召回需要算几亿的物品向量
- 而用户兴趣动态变化，物品特征趋于稳定

增量更新与全量更新

- **全量更新**：今天凌晨，用昨天的数据训练整个神经网络，做 1 epoch 的随机梯度下降。
- **增量更新**：用实时数据训练神经网络，只更新 ID Embedding，锁住全连接层。
- **实际的系统**：
 - **全量更新 & 增量更新** 相结合。
 - 每隔几十分钟，发布最新的用户 ID Embedding，供用户塔在线上计算用户向量。

全量更新:只用一遍数据，训练1 epoch后发布新的用户塔神经网络和物品向量供线上召回使用。对数据流、系统的要求较低

增量更新：实时收集线上数据，做online learning，增量更新ID Embedding参数，发布用户ID Embedding

全量训练的模型更好，而增量训练可以实时捕捉用户的兴趣。

而增量训练中，小时/分钟级别的数据有偏，与全天的数据差别很大。因此，不能只增量训练，而做全量训练的时候也要随机排列数据

其他召回

GeoHash召回

对经纬度进行编码，通过GeoHash召回附近的优质笔记

同城召回

召回同城的内容

作者召回

- 关注作者：用户-关注的作者-最新的笔记
- 交互作者：用户-有交互的作者-最新的笔记
- 相似作者：用户-感兴趣作者-相似的作者-最新的笔记

缓存召回

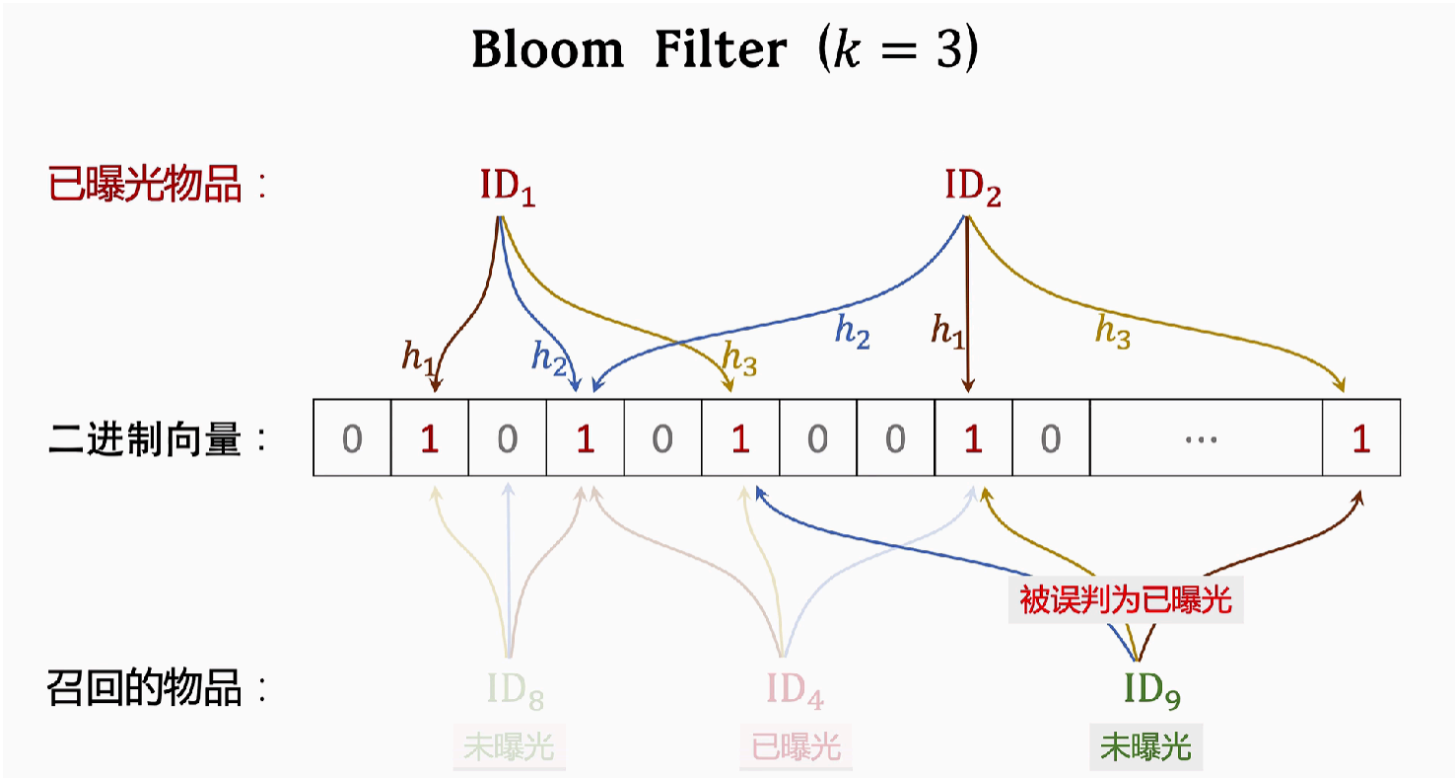
- 复用前n次推荐精排的结果
- 精排前50但是没有曝光的缓存起来,作为召回通道
- 退场机制：被曝光，召回次数或缓存时间达标

曝光过滤问题

不曝光用户已浏览的物品

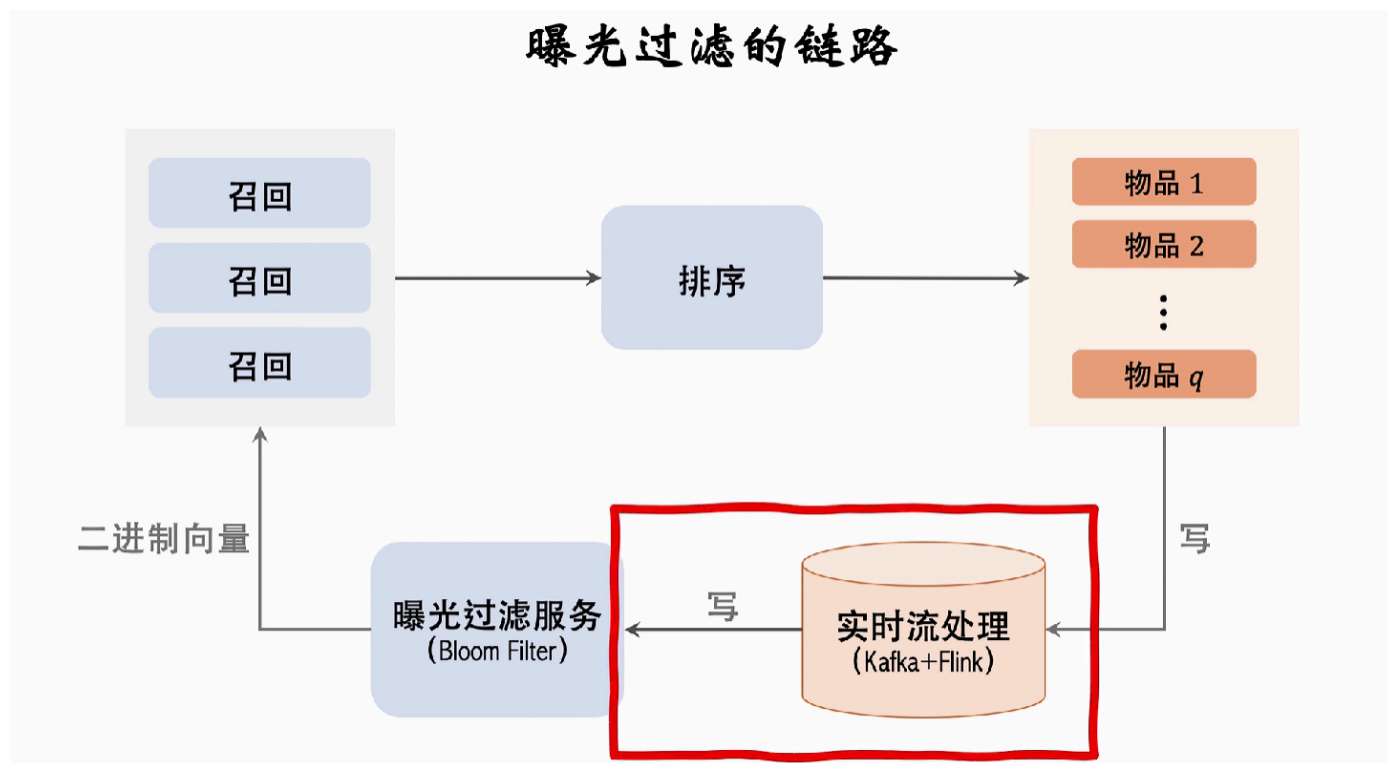
Boom Filter

- 判断一个物品ID是否在已曝光的集合中
- no则一定未曝光,但yes可能曝光(有误差概率)



误伤概率的三个影响因素：

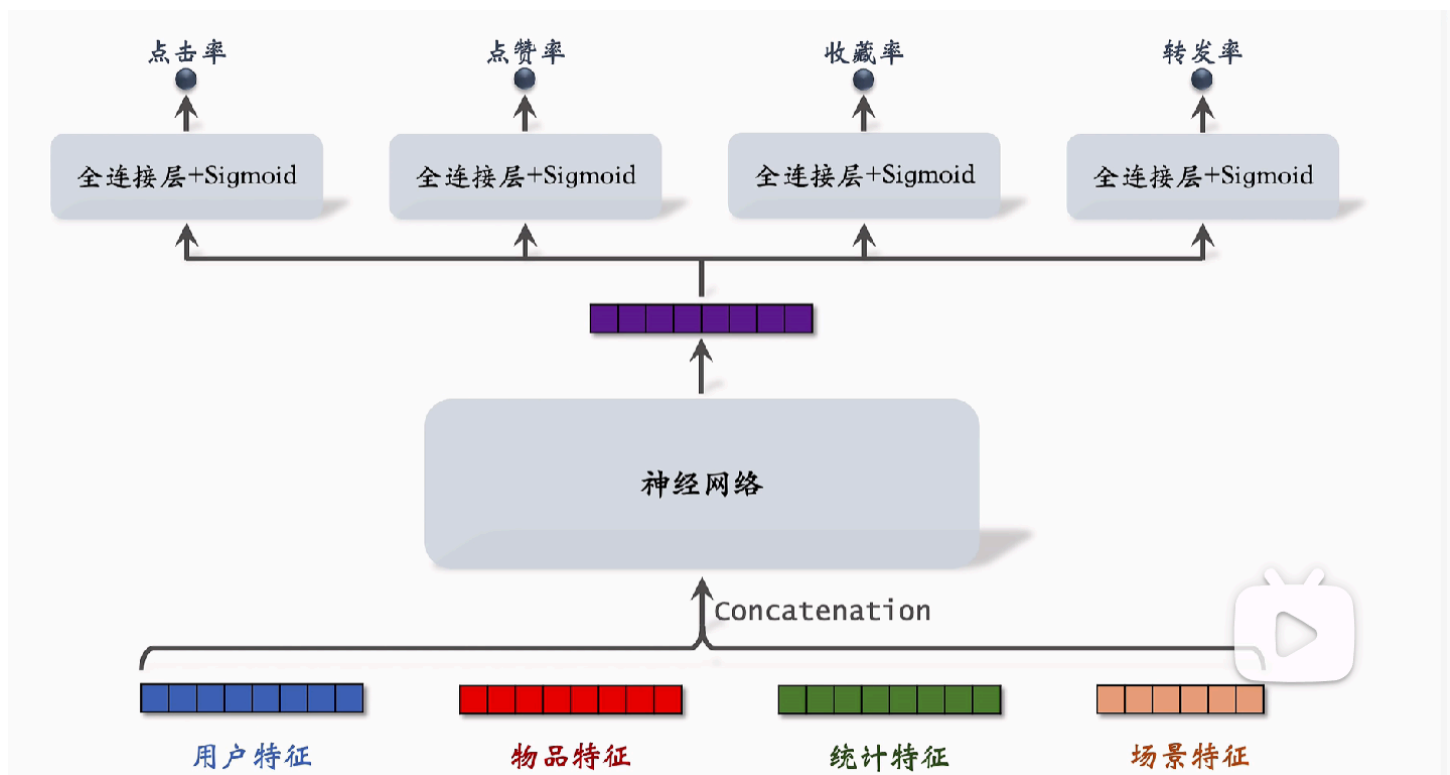
- 物品集合 n
- 二进制向量维度 m
- 哈希函数的个数 k



缺点:只支持添加物品,不支持删除物品。从集合中移除物品无法消除对向量的影响。
每天都需要从集合中移除年龄大于一个月的物品(超龄物品不会被召回无需记录)

多目标模型

指标:笔记的点击率、点赞率、收藏率、转发率



四个神经网络分别输出点击率、点赞率、收藏率、转发率的预估值(0-1)
通过交叉熵损失函数的加权和来恒定预估的准确度，通过梯度下降训练神经网络

实际训练的困难：样本不平衡问题

正样本少，负样本多。

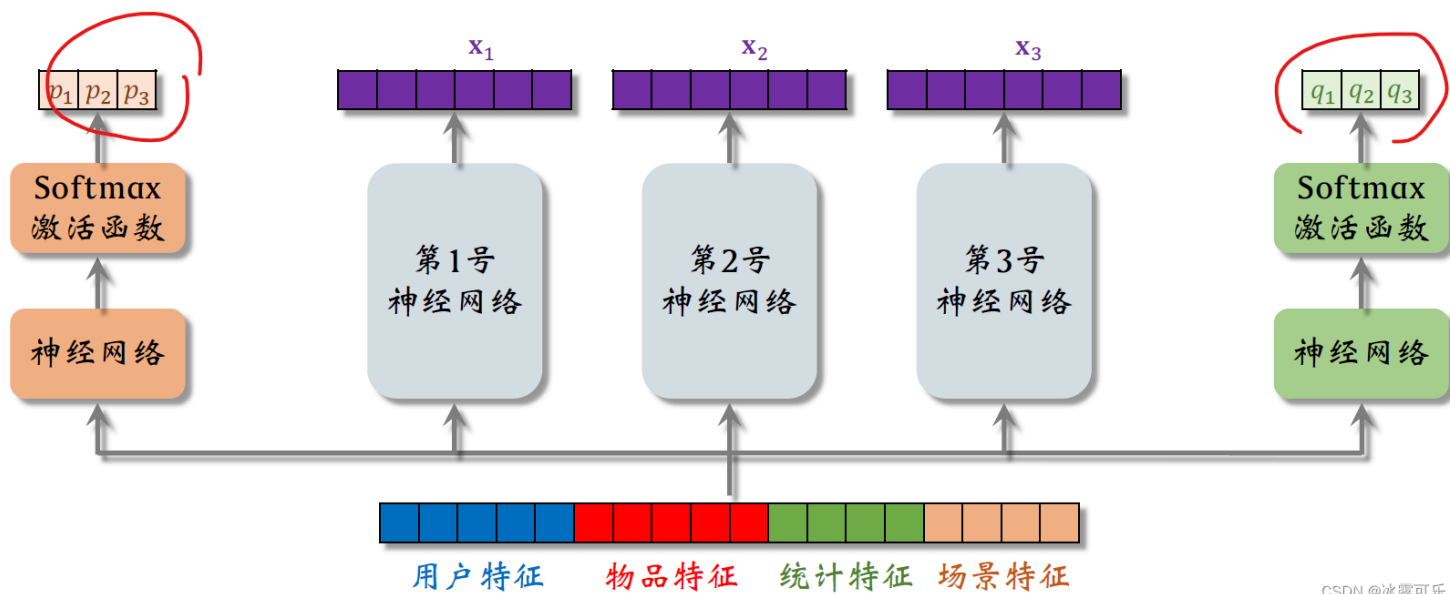
这负样本的数量非常不平衡，较多的负样本用途不大，白白浪费计算资源

解决方案：负样本随机降采样

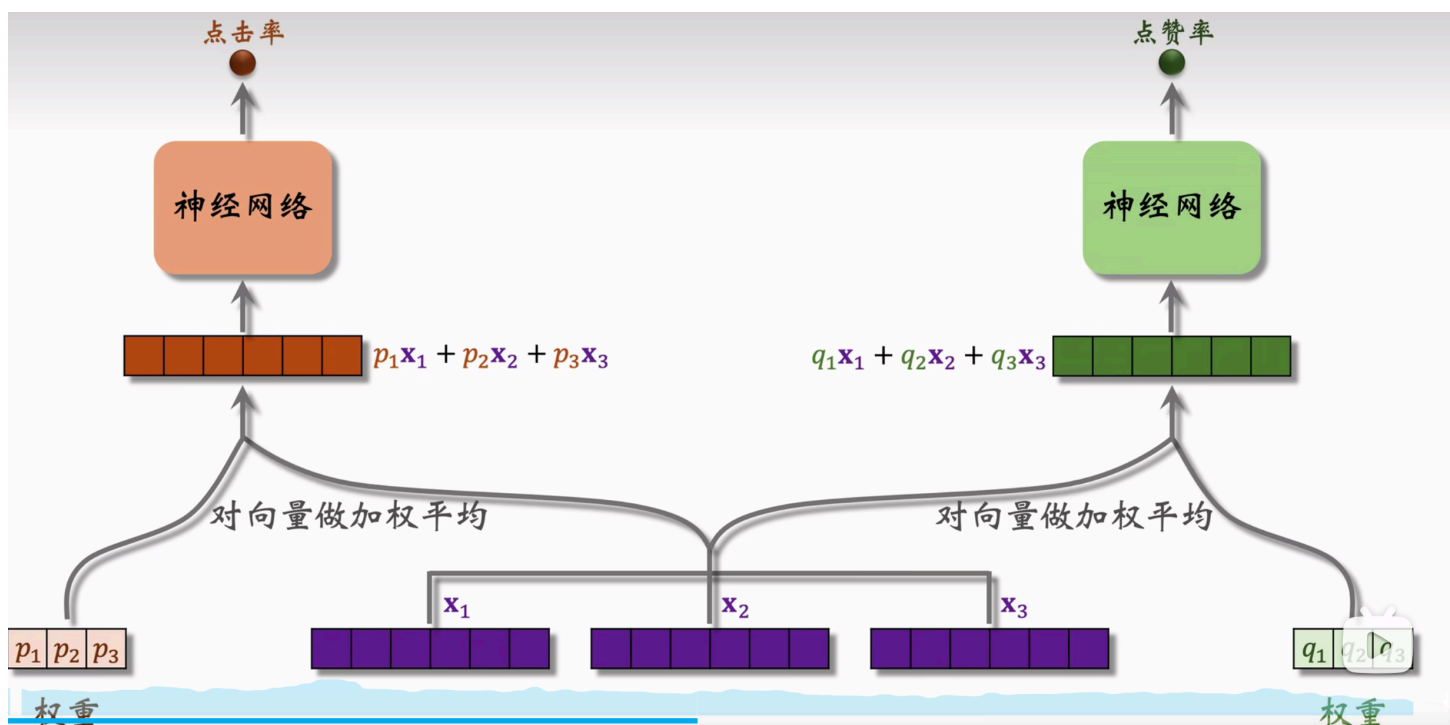
减少样本数量，降低训练的计算代价。

由于降采样，预估值需要校准（借助预估点击率，采样率和正负样本个数）

MMOE



将特征输入三个结构相同，但不共享参数的专家神经网络（实际上不止三个）得到向量 x_1 x_2 x_3 。

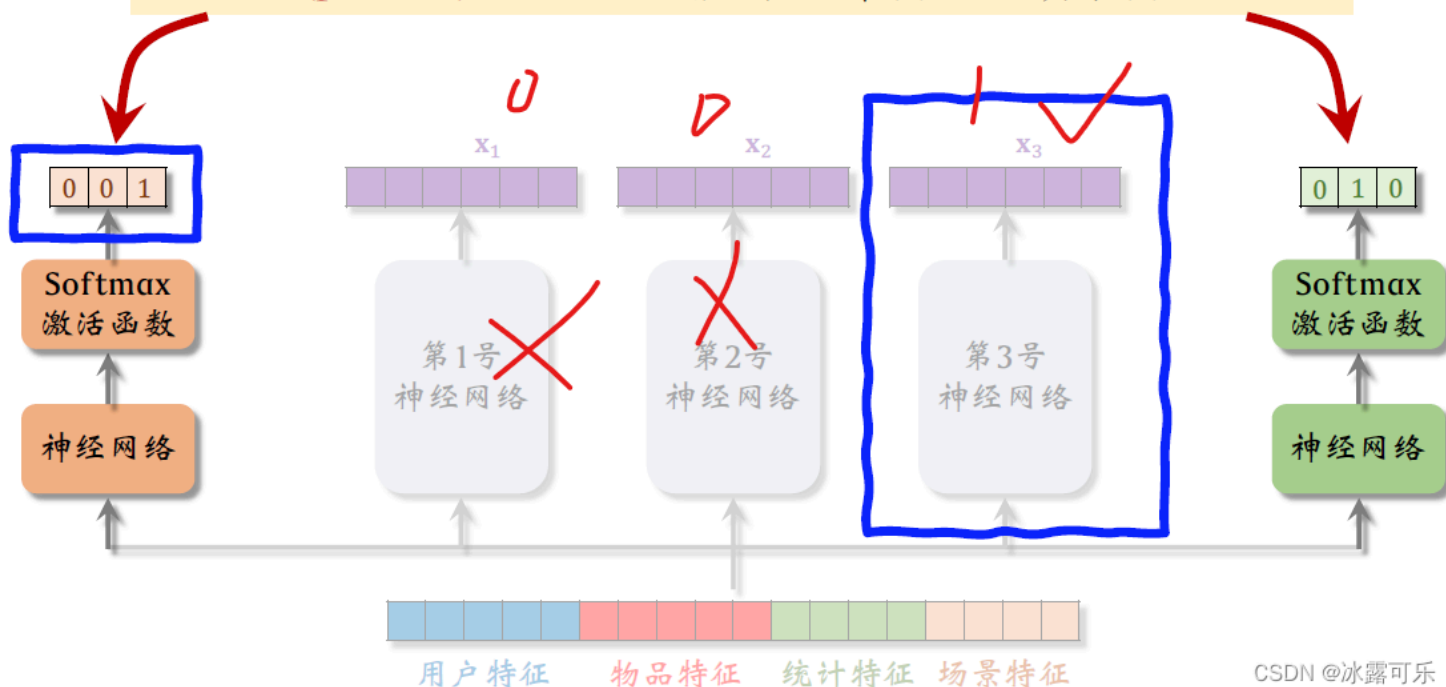


紧接着，将特征向量输入另外的神经网络，并以softmax激活，得到都大于零，而且相加等于一向量的两组元素，记作 $\{p_1, p_2, p_3\}$ 和 $\{q_1, q_2, q_3\}$ 。

用P和Q这两组元素作为权重，对得到的向量 x_1 x_2 x_3 做加权平均，用加权平均得到的向量去预估业务指标（这里是点击率和点赞率，因为有两组权重，假如有十个目标，那么就要用十组权重）。

MMOE存在极化问题

极化(polarize)：Softmax输出值一个接近1，其余接近0。



CSDN @冰露可乐

某一权重过小导致没有让三个专家神经网络的输出融合，而是简单使用了一个专家。

通过dropout解决极化问题

解决极化现象的一种方法是dropout

在训练的过程中被softmax输出使用dropout

对于softmax输出的N个数值（N为专家的个数），每个数值都有10%的概率被mask

也就是说在训练的过程中，每个专家被丢弃的概率都是10%。

强迫每个任务根据部分专家做预测

融合预估分数

简单的加权和

$$p_{\text{click}} + w_1 \cdot p_{\text{like}} + w_2 \cdot p_{\text{collect}} + \dots$$

点击率乘以其他项的加权和

$$p_{\text{click}} \cdot (1 + w_1 \cdot p_{\text{like}} + w_2 \cdot p_{\text{collect}} + \dots)$$

国内某短视频APP的融分公式

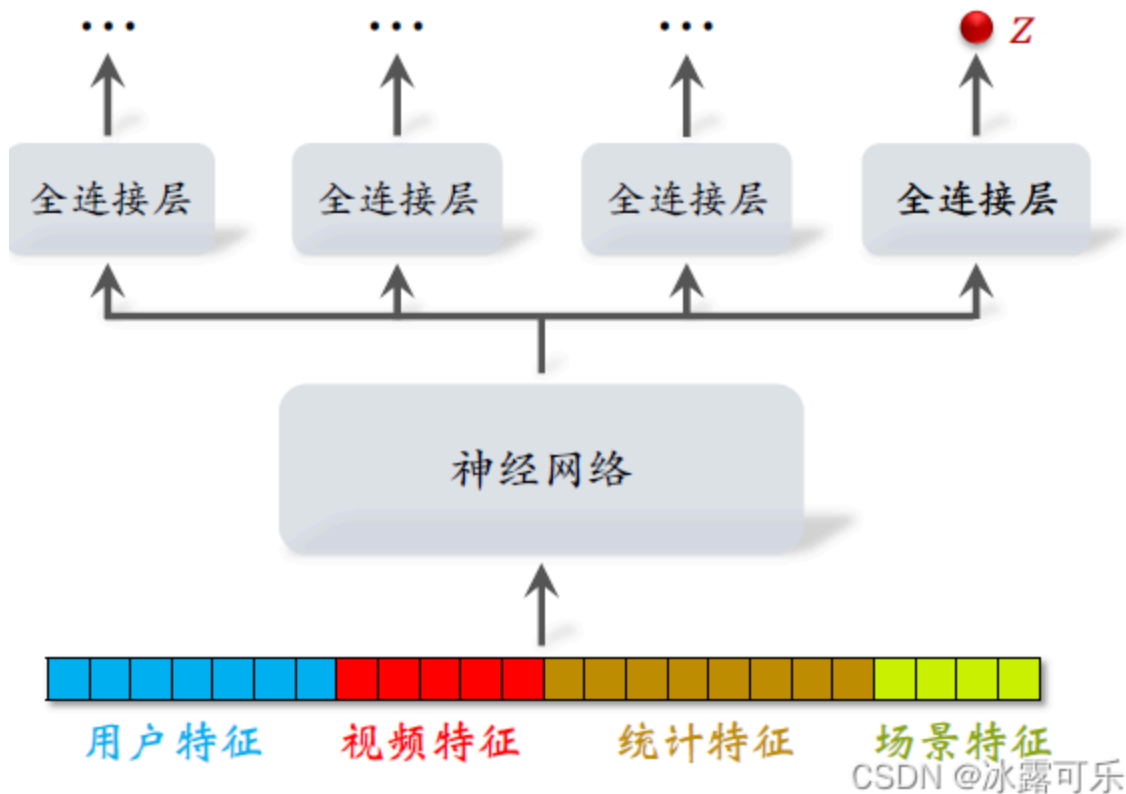
- 根据预估时长 p_{time} ，对 n 篇候选视频做排序。
- 如果某视频排名第 r_{time} ，则它得分 $\frac{1}{r_{\text{time}}^{\alpha} + \beta}$ 。
- 对点击、点赞、转发、评论等预估分数做类似处理。
- 最终融合分数：

$$\frac{w_1}{\underline{r_{\text{time}}^{\alpha_1}} + \beta_1} + \frac{w_2}{\underline{r_{\text{click}}^{\alpha_2}} + \beta_2} + \frac{w_3}{\underline{r_{\text{like}}^{\alpha_3}} + \beta_3} + \dots$$

视频

图文笔记和视频排序的依据不同
两个重要指标：播放时长、完播率

Youtube对视频观看时长的方法：



将特征输入多层神经网络，被所有任务共享。

神经网络上有多个全连接层，每个全连接层对应一个目标比如点击、点赞、收藏等

这里只关心对播放时长的预估，对于全连接层得到的 Z ，做sigmoid后与我们定义的 Y (观看时长 T 进行类sigmoid变换)，最小化交叉熵。

$$\bullet y = \frac{t}{1+t}$$

如果 $p = y$ ，那么 $\exp(z) = t$ 。



$$p = \frac{\exp(z)}{1 + \exp(z)}$$

完播

回归

让预估播放率来拟合 y

二元分类

定义完播的意义，用二元分类器预估完播率

在实践中，我们不能把预估的完播率直接用到融分公式中，

因为，视频越长，完播率就越低。所以应该对完播率做一定调整

线上预估完播率，然后做调整：

$$\underline{p_{\text{finish}}} = \frac{\text{预估完播率}}{f(\text{视频长度})}$$

把 p_{finish} 作为融分公式中的一项。

排序的特征

1. 用户画像

- 用户ID
- 人口统计学属性：性别、年龄
- 账号信息：新老、活跃度
- 感兴趣的类目、关键词

2. 物品画像

- 物品ID (embedding)
- 发布时间
- GeoHash
- 标题、类目、关键词等离散特征
- 字数、图片书、清晰度等自带属性

3. 用户统计特征

- 用户最近一段时间的曝光时点击数等
- 按照图文\视频分桶
- 按照笔记类分桶

4. 笔记统计特征

- 笔记最近一段时间的曝光点击等
- 按照性别、用户年龄等分桶
- 作者特征

5. 场景特征

- 用户定位、城市
- 当前时刻
- 是否周末节假日等
- 手机品牌价格等

特征处理

- 离散特征：做embedding
- 连续特征：

1. 做分桶，变成离散特征
2. 其他变换，曝光数点击数等做 $\log(1+x)$ /转换为点击率点赞率等并做平滑

粗排

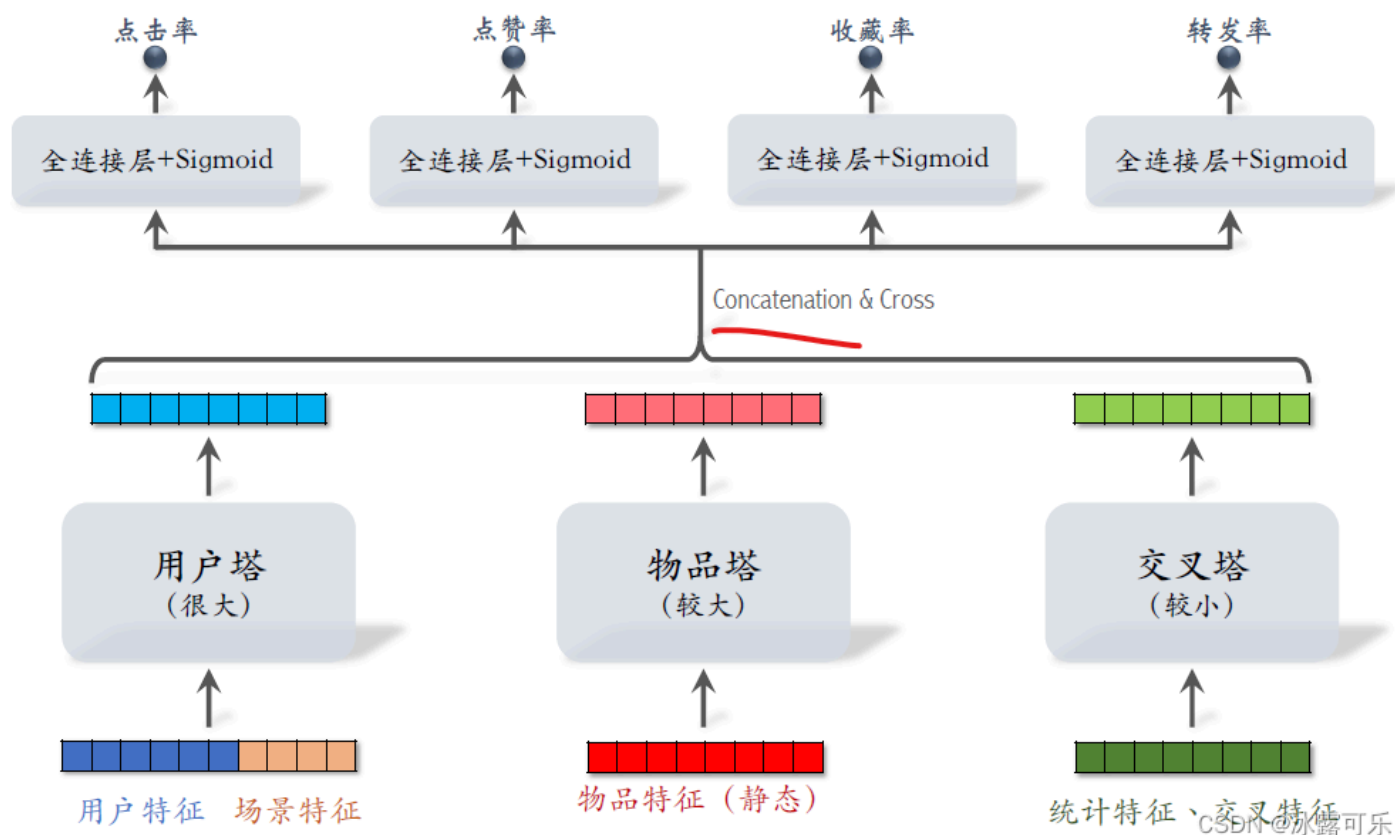
相比于精排，打分的数量更多，推理代价更小，准确性不高

精排 vs 双塔

精排：先对特征做concatenation再输入神经网络（前期融合）。线上推理代价大n个物品推理n次

双塔：用户、物品分别输入不同神经网络（后期融合）。线上计算小，只需要做一次线上推理

粗排 三塔模型



每次粗排，需要取1个用户和n个物品的特征

- 用户塔，只做一次推理
- 物品塔，未命中缓存时需要做推理(物品特征比较稳定，所以预先计算并缓存)
- 交叉塔，必须做n次推理，因为统计特征会实时变化

- 上层网络，做n次推理得到对应指标

重排——精排后的多样性后处理

物品相似度度量

基于物品属性标签

类目、品牌、关键词

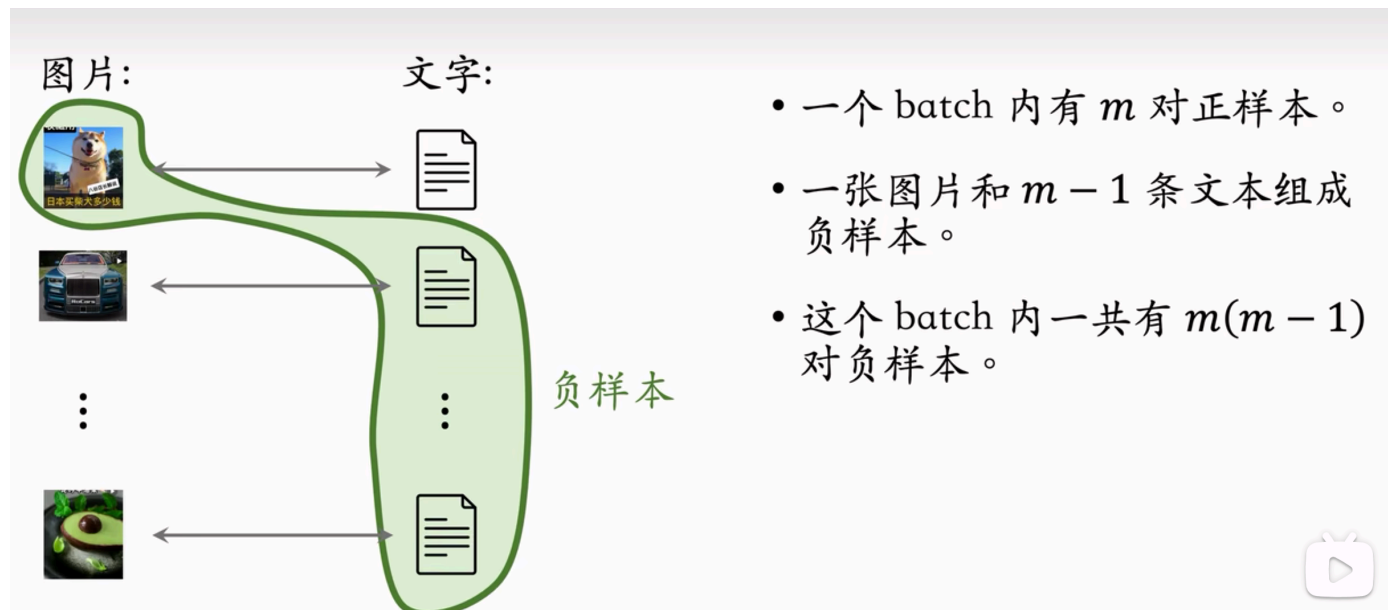
根据一级类目、二级类目、品牌计算相似度

基于物品向量表征

双塔模型得到的物品表征：效果不好，存在长尾效应

基于图文内容的物品标准，利用CLIP

- 思路：对于图片——文本二元组，预测图文是否匹配
- 优势：无需人工标注。



正负样本选取类似双塔模型

MMR

多样性

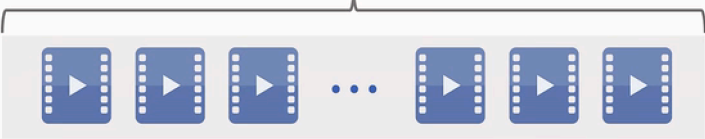
- 精排对物品打分后的分数 $\text{reward}_1, \dots, \text{reward}_n$
- 第 i 个和第 j 个物品的相似度为 $\text{sim}(i, j)$
- 从 n 个物品中选取 k 个，既要有高精排分数也要有多样性

多样性算法

选中的物品（记作 $\mathcal{S}$ ）



未选中的物品（记作 $\mathcal{R}$ ）



- 计算集合 $\mathcal{R}$ 中每个物品 i 的 Marginal Relevance 分数：

$$\text{MR}_i = \theta \cdot \text{reward}_i - (1 - \theta) \cdot \max_{j \in \mathcal{S}} \text{sim}(i, j).$$

通过未选中物品与已选中物品的相似性来确定是否加入集合

MMR的算法流程

- 做初始化，已选中为空，未选中为全集
- 选择精排分数最高的物品移入选中集合
- 做 $k-1$ 轮循环
 - i. 计算未选中物品中的MR分数
 - ii. 选出分数最高的物品移入选中集合

问题

已选中物品过多， sim 接近于1

解决方法

设置一个滑动窗口，用窗口内的物品代替公式中已选中的物品

重排规则

1. 最多连续出现k篇某种笔记

例：最多连续出现5篇视频笔记/图文笔记

每k篇笔记中最多出现一篇某种笔记

例：每9篇笔记中最多出现一篇运营推广笔记

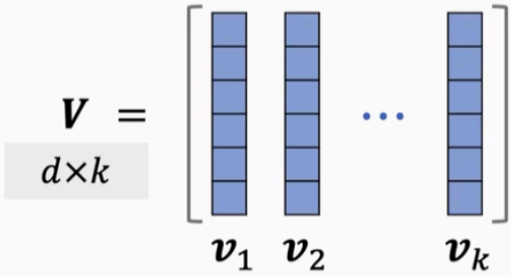
前t篇笔记最多出现k篇某种笔记

排名前t篇最容易看到，对用户体验最重要

DPP多样性算法

超平行体

- 给定 k 个物品，把它们表征为单位向量 $\boldsymbol{v}_1, \dots, \boldsymbol{v}_k \in \mathbb{R}^d$ 。 ($d \geq k$)
- 用超平行体的体积衡量物品的多样性，体积介于 0 和 1 之间。
- 如果 $\boldsymbol{v}_1, \dots, \boldsymbol{v}_k$ 两两正交（多样性好），则体积最大化， $\text{vol} = 1$ 。
- 如果 $\boldsymbol{v}_1, \dots, \boldsymbol{v}_k$ 线性相关（多样性差），则体积最小化， $\text{vol} = 0$ 。



$V = \begin{bmatrix} | & | & & | \\ v_1 & v_2 & \dots & v_k \\ | & | & & | \end{bmatrix}$

$d \times k$

- 给定 k 个物品，把它们表征为单位向量 $v_1, \dots, v_k \in \mathbb{R}^d$ 。 ($d \geq k$)
- 把它们作为矩阵 $V \in \mathbb{R}^{d \times k}$ 的列。
- 设 $d \geq k$ ，行列式与体积满足：

$$\det(V^T V) = \text{vol}(\mathcal{P}(v_1, \dots, v_k))^2.$$
- 因此，可以用行列式 $\det(V^T V)$ 衡量向量 $v_1, \dots, v_k$ 的多样性。

精排给 n 个物品打分得到： $\text{reward}_1, \dots, \text{reward}_n$

n 个物品的向量表征 $v_1, \dots, v_n$

从 n 个物品中选出 k 个物品，组成集合 S ，要求

- 价值大， reward 之和越大越好
- 多样性好： k 个向量组成的超平行体的体积越大越好， 因为 $k < d$ ， 所以此时可以用行列式 $V^T V$ 来计算体积

DPP： 用行列式衡量多样性

$$\underset{S: |S|=k}{\operatorname{argmax}} \quad \theta \cdot \left(\sum_{j \in S} \text{reward}_j \right) + (1 - \theta) \cdot \log \det(V_S^T V_S).$$

第一项为 reward 之和， 第二项衡量多样性

用贪心算法近似求解公式

和MMR一样存在着 S 过大时的过相似问题

解决方法： 滑动窗口， 只考虑最近加入集合的

涨指标的方法

要涨什么指标

最核心的指标 日活用户数 (DAU)， 留存

工业界最常用的指标LT7,LT30

LT： 包括今天在内的未来7天/30天有多少天登录了APP

涨指标的方法

- 改进召回模型
- 改进精排和粗排模型
- 提升召回、粗排、精排的多样性
- 特殊对待新用户、低活用户
- 利用关注、转发、评论这三种交互行为

改进召回模型

改进双塔模型

1. 优化正样本、负样本

简单正样本——有点击的二元组

简单负样本——随机组合的二元组

困难负样本——排序靠后的二元组

2. 改进神经网络结构

- 在用户塔、物品塔中用DCN代替全连接网络（FCN）
- 在用户塔中使用用户行为序列（last-n）
- 用多向量模型代替单向量模型，用户塔输出很多向量，通过用户塔的向量分别与物品塔生成的向量计算相似度进行指标预估

3. 改进物品训练方法

- 结合二分类、batch内负采样
- 使用自监督学习方法，改善冷门物品的embedding学习

改进I2I模型

1. 综合使用ItemCF及其变体

线上同时使用ItemCF、Online ItemCF、Swing、Online Swing模型，各分配一定份额

2. 基于物品向量表征，计算物品相似度

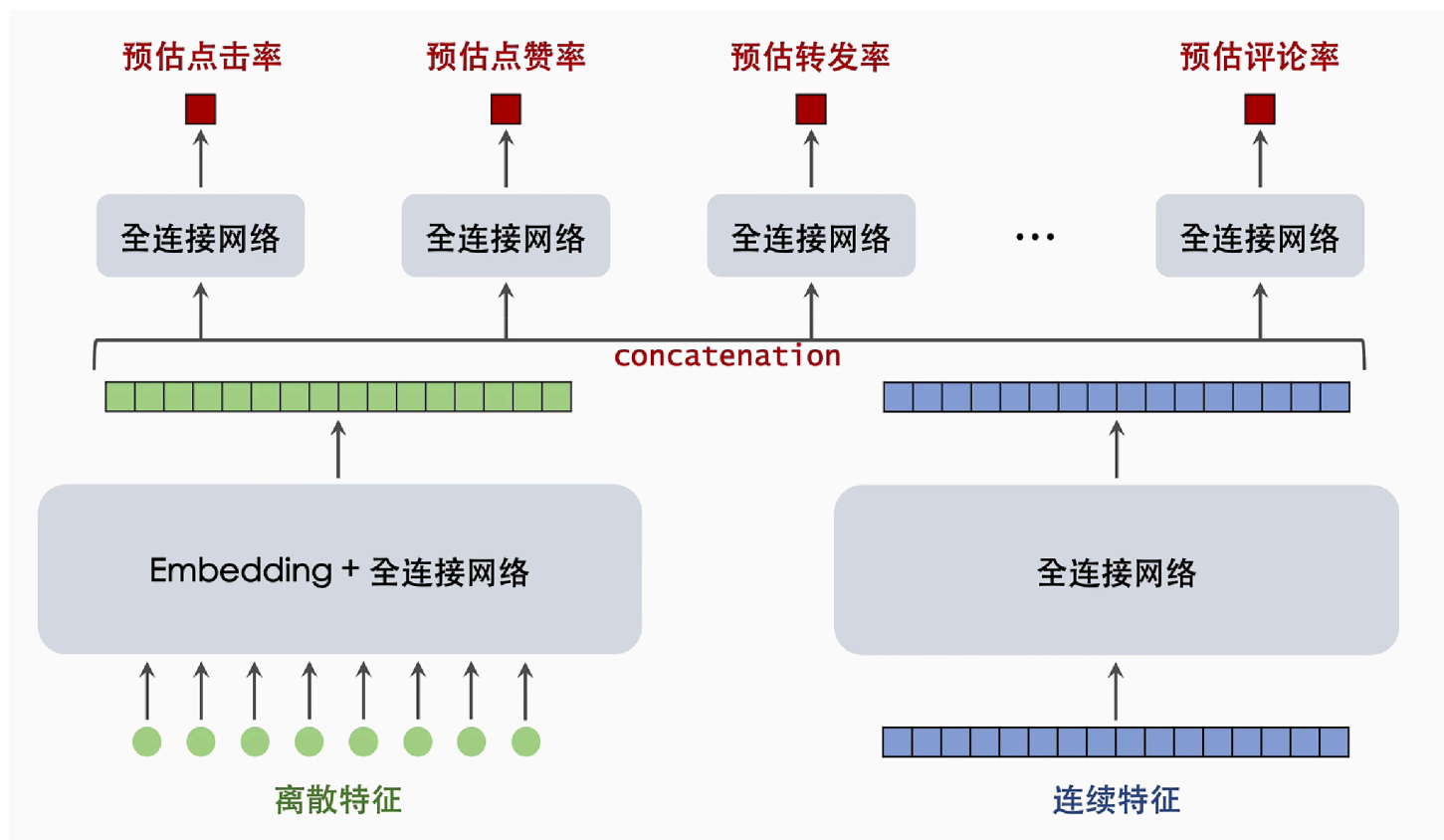
采用双塔、图神经网络计算表征

采用小众召回模型

- U2U2I (user $\rightarrow$ user $\rightarrow$ item) : 已知用户 u_1 与 u_2 相似, 且 u_2 喜欢物品 i , 那么给用户 u_1 推荐物品 i 。
- U2A2I (user $\rightarrow$ author $\rightarrow$ item) : 已知用户 u 喜欢作者 a , 且 a 发布物品 i , 那么给用户 u 推荐物品 i 。
- U2A2A2I (user $\rightarrow$ author $\rightarrow$ author $\rightarrow$ item) : 已知用户 u 喜欢作者 a_1 , 且 a_1 与 a_2 相似, a_2 发布物品 i , 那么给用户 u 推荐物品 i 。

改进排序模型

精排模型改进



1. 改进基座

基座的输入为连续特征和离散特征, 输出一个向量作为多目标预估的输入

- 基座加宽加深, 计算量变大, 预测更精确

- 做自动特征交叉，bilinear等
- 特征工程，添加统计特征、多模态特征等

2. 改进多目标预估

模型基于底座输出的向量，同时预估点击率等多个目标

- 增加新的预估目标，并把预估结果写入统和公式
- MME, PLE等可能有效
- 纠正 position bias可能有效

粗排模型改进

粗排的打分量可能是精排的数十倍，所以粗排必须足够快

粗精排一致性建模

- pointwise蒸馏， y 为用户真实行为， p 为精排预估。用 $(y+p)/2$ 做粗排目标
- pairwise或listwise蒸馏， k 个物品精排后做learning to rank，让粗排拟合物品序
可以提升核心指标 但是精排出bug会污染数据

用户行为序列建模

1. 增加序列长度 但是会增加计算成本
2. 筛选行为序列 用类目、物品向量表征聚类
3. 对物品行为序列中的物品，使用ID之外的一些特征
采用最多的方法——增加序列长度后筛选变短输入DIN

在线学习

在线学习的资源消耗

- 线上有 m 个模型，其中 1 个是 holdout，1 个是推全的模型， $m-2$ 个测试的新模型。
- 每套在线学习的机器成本都很大，因此 m 数量很小，制约模型开发迭代的效率。
- 在线学习对指标的提升巨大，但是会制约模型开发迭代的效率。

老汤模型

存在问题

老模型训练时间更久，数据更多。新模型训练时间短，对比困难

1. 如何判断新模型更优

新老模型给均随机初始化全连接层，用n天的数据训练看结果

2. 如何快速追平老模型

- 尽可能复用老模型训练好的embedding层，而非初始化embedding层
- 用老模型做teacher，蒸馏新模型

多样性

总结：提升多样性

- 精排：结合兴趣分数和多样性分数做排序；做规则打散。
- 粗排：只用兴趣分数选出部分物品；结合兴趣分数和多样性分数选出部分物品。
- 召回：往双塔模型的用户向量添加噪声；对用户行为序列做非均匀随机抽样（对双塔和 U2I2I 都适用）。
- 兴趣探索：保留少部分的流量给非个性化推荐。

特殊对待特殊人群

1. 召回：构造特殊内容池

- 根据物品获得的交互次数、交互率选择优质物品
- 做因果推断，判断物品对人群留存率的贡献，按照贡献值选择物品

2. 排序策略

- 排除低质量物品，保护特殊人群的体验
- 对于特殊用户人群使用特殊的融分公式

3. 排序模型

- 结合大模型和小模型，小模型拟合大模型的残差，对大模型做校准
- 只用一个模型，模型有多个expert

利用交互行为

关注

- 留存价值：利用排序策略提升关注量，构造促关注的内容池和召回通道
- 发布价值：利用排序策略帮助低粉新作者涨粉
- 隐式关注关系：构造U2A2I召回

转发

判断哪些用户是站外KOL，利用转发吸引站外流量

评论

- 发布价值：促进新物品获得评论，提升作者积极性
- 留存价值：给喜欢讨论的用户创造更多评论机会
- 鼓励高质量评论用户进行评论